PROGRAMACIÓN COMPETITIVA · ICPC

Algoritmario ICPC

Referencia rápida de algoritmos para competencia

24	5	1,200
ALGORITMOS	CATEGORÍAS	LÍNEAS DE CÓDIGO

EDICIÓN GENERADA EL 13 DE AGOSTO DE 2026

Índice

Las categorías y los algoritmos están enlazados. Haz clic en cualquier entrada para ir directamente a su página.

SECCIÓN 01

Data Structures

4 algoritmos

bint

DATA STRUCTURES · C++ · 84 LÍNEAS

CÓDIGO · bint

```cpp
#include <bits/stdc++.h>
const int base {1'000'000'000};

class Bint {
public:
    Bint() = default;
    Bint(int num) : d(1, num) {};
    Bint(std::string& num) {
        int n{std::ssize(num)};
        for (int i{n}; i > 0; i -= 9) {
            if (i >= 9) {
                auto s = num.substr(i - 9, 9);
                d.push_back(std::stoi(s));
            } else {
                auto s = num.substr(0, i);
                d.push_back(std::stoi(s));
            }
        }
    }

    Bint operator+(Bint& o) {
        Bint bint{};
        auto& ans{bint.d};
        int carry{};
        for (int i{}; i < std::max(size(), o.size()); ++i) {
            auto x{(i < size()) ? d[i] : 0};
            auto y {(i < o.size()) ? o[i] : 0};
            auto v {(x + y + carry)};
            ans.push_back(v % base);
            carry = (v >= base);
        }
        if (carry) ans.push_back(carry);
        bint.trim();
        return bint;
    }

    Bint operator*(int b) {
        Bint bint{};
        auto& ans {bint.d};
        long long c{};
        for (int i{}; i < size(); ++i) {
            long long x {(d[i] * 1ll * b) + c};
            ans.push_back(x % base);
            c = (x / base);
        }
        while (c) {
            ans.push_back(c % base);
            c /= base;
        }
        bint.trim();
        return bint;
    }
    bool operator==(Bint& o) {
        if (size() != o.size()) return false;
        for (int i{}; i < size(); ++i) {
            if (d[i] != o[i]) return false;
        }
        return true;
    }
    bool operator<(Bint& o) {
        if (size() != o.size()) return size() < o.size();
        for (int i{size() - 1}; i >= 0; --i) {
            if (d[i] != o[i]) return d[i] < o[i];
        }
        return false;
    }
```

CÓDIGO · bint

```
67
68      std::string toString() {
69          if (d.empty()) return "0";
70          std::string s{std::format("{}", d.back())};
71          s.reserve(9 * size());
72          for (int i{size() - 2}; i >= 0; --i) {
73              s += std::format("{:09d}", d[i]);
74          }
75          return s;
76      }
77  private:
78      std::vector<int> d{};
79      int operator[](int i) { return d[i]; }
80      int size() { return std::ssize(d); }
81      void trim() {
82          while (size() > 1 && d.back() == 0) d.pop_back();
83      }
84  };
```

bit

DATA STRUCTURES · C++ · 46 LÍNEAS

 # CÓDIGO · bit

```cpp
#include <bits/stdc++.h>

class Bit {
public:
    Bit (std::vector<long long>& a) :
        n{std::ssize(a) + 1}, ft(n) {
            for (int i{1}; i < n; ++i) {
                ft[i] += a[i - 1];
                if (parent(i) < n) ft[parent(i)] += ft[i];
            }
        }
    auto sum(int i) {
        long long ans{};
        while (i) {
            ans += ft[i];
            i = child(i);
        }
        return ans;
    }
    auto sumQuery(int l, int r) {
        return sum(r) - sum(l - 1);
    }
    void update(int i, long long delta) {
        while (i < n) {
            ft[i] += delta;
            i = parent(i);
        }
    }
    int find(long long k) {
        int i{};
        int b = std::bit_floor(static_cast<unsigned int>(n));
        for (int j{b}; j; j >>= 1) {
            if (i + j < n && ft[i + j] < k) {
                k -= ft[i + j];
                i += j;
            }
        }
        return i + 1;
    }
private:
    int n{};
    std::vector <long long> ft{};
    int lsb    (int i) { return i&(-i); }
    int parent (int i) { return i + lsb(i); }
    int child  (int i) { return i - lsb(i); }
};
```

dsu

DATA STRUCTURES · C++ · 30 LÍNEAS

CÓDIGO · dsu

```cpp
#include <bits/stdc++.h>

class UF {
public:
    UF (int size)
        : n{size}, c{n}, p(n), sz(n, 1) {
            std::iota(p.begin(), p.end(), 0);
        }
    int find (int i) {
        while (p[i] != i) i = p[i] = p[p[i]];
        return i;
    }
    bool same(int i, int j) {
        return find(i) == find(j);
    }
    bool unite(int i, int j) {
        i = find(i); j = find(j);
        if (i == j) return false;
        if (sz[i] < sz[j]) std::swap(i, j);
        p[j] = i;
        sz[i] += sz[j];
        --c;
        return true;
    }
    int size(int i) { return sz[find(i)]; }
    int components() { return c; }
private:
    int n{}, c{};
    std::vector<int> p{}, sz{};
};
```

segmentTree

DATA STRUCTURES · C++ · 61 LÍNEAS

 # CÓDIGO · segmentTree

```cpp
#include <bits/stdc++.h>

class SegmentTree {
    using T = int;
public:
    SegmentTree (const std::vector<T>& a)
        : n{std::ssize(a)}, tree(n * 4, kdefault), lazy(n * 4, klazy) {
            build(0, 0, n - 1, a);
        }
    void increment (int l, int r, T delta);
    T query        (int l, int r);
private:
    int n{};
    const T kdefault{}, klazy{};
    std::vector<T> tree{}, lazy{};

    T operation (T a, T b) { return std::max(a, b); }
    int left  (int node)     { return (node << 1) + 1; }
    int right (int node)     { return (node << 1) + 2; }
    int mid   (int a, int b) { return (a + b) / 2; }

    void build(int node, int l, int r, const std::vector<T>& a) {
        if (l == r) {
            tree[node] = a[l];
            return;
        }
        build(left(node), l, mid(l, r), a);
        build(right(node), mid(l , r) + 1, r, a);
        tree[node] = operation(tree[left(node)], tree[right(node)]);
    }
    void increment(int node, int l, int r, int leftQuery, int rightQuery, T delta) {
        if (r < leftQuery || rightQuery < l) return;
        propagate(node, l, r);
        if (leftQuery <= l && r <= rightQuery) {
            lazy[node] += delta;
            propagate(node, l, r);
            return;
        }
        increment(left(node), l, mid(l, r),      leftQuery, rightQuery, delta);
        increment(right(node), mid(l, r) + 1, r, leftQuery, rightQuery, delta);
        tree[node] = operation(tree[left(node)], tree[right(node)]);
    }
    T query(int node, int l, int r, int leftQuery, int rightQuery) {
        if (r < leftQuery || rightQuery < l) return kdefault;
        propagate(node, l, r);
        if (leftQuery <= l && r <= rightQuery) return tree[node];
        return operation (
            query(left(node), l, mid(l, r), leftQuery, rightQuery),
            query(right(node), mid(l, r) + 1, r, leftQuery, rightQuery)
        );
    }
    void propagate (int node, int l, int r) {
        if (lazy[node] == klazy) return;
        tree[node] += lazy[node];
        if (l != r) {
            lazy[left(node)]  += lazy[node];
            lazy[right(node)] += lazy[node];
        }
        lazy[node] = klazy;
    }
};
```

SECCIÓN 02

Graph

9 algoritmos

bellmanford

GRAPH · C++ · 29 LÍNEAS

CÓDIGO · bellmanford

```cpp
#include <bits/stdc++.h>

dis[start] = 0;
for (int i{}; i < n - 1; ++i) {
    for (int node{}; node < n; ++node) {
        if (dis[node] == INF) continue;
        for (auto [u, w] : adj[node]) {
            dis[u] = std::min(dis[u], dis[node] + w);
        }
    }
}
std::vector<int> ciclosos(n), q{};
for (int node{}; node < n; ++node) {
    if (dis[node] == INF) continue;
    for (auto [u, w] : adj[node]) {
        if (dis[node] + w < dis[u]) {
            ciclosos[u] = true;
            q.push_back(u);
        }
    }
}
for (int j{}; j < std::ssize(q); ++j) {
    auto node {q[i]};
    for (auto [u, w] : adj[node]) {
        if (ciclosos[u]) continue;
        ciclosos[u] = true;
        q.push_back(u);
    }
}
```

bridgesArticulation

GRAPH · C++ · 35 LÍNEAS

 # CÓDIGO · bridgesArticulation

```cpp
#include <bits/stdc++.h>
int n{}, m{};

// Undirected edges {vertex, id}
std::vector<std::vector<std::pair<int,int>>> adj(n);

std::vector<int> bridges(m), articulation(n);
int root{}, childrenRoot{};

std::vector<int> pre(n), low(n);
int cont{};
auto dfs = [&](auto& self, int node, int parentEdge) -> void {
    pre[node] = low[node] = ++cont;
    for (auto [u, id] : adj[node]) {
        if (id == parentEdge) continue;
        if (pre[u] == 0) {
            // forward/tree edge
            self(self, u, id);
            if (node == root) ++childrenRoot;
            if (low[u] > pre[node]) bridges[id] = true;
            if (low[u] >= pre[node]) articulation[node] = true;
            low[node] = std::min(low[u], low[node]);
        } else {
            // back edge
            low[node] = std::min(low[node], pre[u]);
        }
    }
};

for (int i{}; i < n; ++i) {
    if (pre[i]) continue;
    root = i; childrenRoot = 0;
    dfs(dfs, i, -1);
    articulation[i] = (childrenRoot > 1);
}
```

cycleDetection

GRAPH · C++ · 34 LÍNEAS

 # CÓDIGO · cycleDetection

```cpp
#include <bits/stdc++.h>

enum Colors {none, explored, visited};
std::vector<int> vis(n);
auto hasCycle = [&](auto& self, int node) -> bool {
    vis[node] = explored;
    for (auto u : adj[node]) {
        if (vis[u] == none) {
            if (self(self, u)) return true;
        } else if (vis[u] == explored) return true;
    }
    vis[node] = visited;
    return false;
};

std::vector<int> q{};
q.reserve(n);
for (int i{}; i < n; ++i) {
    if (inDegree[i] == 0) {
        q.push_back(i);
    }
}
for (int j{}; j < std::ssize(q); ++j) {
    auto node {q[j]};
    for (auto u : adj[node]) {
        --inDegree[u];
        if (inDegree[u] == 0) {
            q.emplace_back(u);
        }
    }
}
if (std::ssize(q) != n) {
    std::cout << "There is a cycle.\n";
}
```

dijkstra

GRAPH · C++ · 26 LÍNEAS

CÓDIGO · dijkstra

```cpp
#include <bits/stdc++.h>

const long long INF = 5e18;
std::vector<int> dis(n, INF);

using T = std::pair<long long, int>;
std::priority_queue<T, std::vector<T>, std::greater<T>> pq{};

dis[start] = 0;
pq.emplace(dis[start], start);

std::vector<std::vector<std::pair<int,int>>> path(n);

while (!pq.empty()) {
    auto [cost, node] {pq.top()}; pq.pop();
    if (cost > dis[node]) continue;
    for (auto [u, w] : adj[node]) {
        if (dis[node] + w < dis[u]) {
            dis[u] = dis[node] + w;
            pq.emplace(dis[u], u);
            path[u] = {std::pair{node, w}};
        } else if (dis[node] + w == dis[u]) {
            path[u].emplace_back(node, w);
        }
    }
}
```

dinic

GRAPH · C++ · 341 LÍNEAS

CÓDIGO · dinic

```cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;

struct FlowEdge {
    int from;
    int to;
    ll cap;
    ll flow;

    FlowEdge(int from, int to, ll cap)
        : from(from), to(to), cap(cap), flow(0) {}
};

struct Dinic {
    const ll INF = (ll)4e18;

    int n;
    int source;
    int sink;

    vector<FlowEdge> edges;
    vector<vector<int>> adj;

    vector<int> level;
    vector<int> ptr;

    Dinic(int n, int source, int sink)
        : n(n), source(source), sink(sink) {

        adj.resize(n);
        level.resize(n);
        ptr.resize(n);
    }

    // Directed edge: from -> to with capacity cap
    void add_edge(int from, int to, ll cap) {
        int id = (int)edges.size();

        // Original edge
        edges.emplace_back(from, to, cap);

        // Residual edge
        edges.emplace_back(to, from, 0);

        adj[from].push_back(id);
        adj[to].push_back(id + 1);
    }

    // Builds the level graph
    bool bfs() {
        fill(level.begin(), level.end(), -1);

        queue<int> q;

        level[source] = 0;
        q.push(source);

        while (!q.empty()) {
            int v = q.front();
            q.pop();

            for (int id : adj[v]) {
                FlowEdge& e = edges[id];
```

CÓDIGO · dinic

```
                // No residual capacity
                if (e.cap - e.flow <= 0) {
                    continue;
                }

                // Already visited
                if (level[e.to] != -1) {
                    continue;
                }

                level[e.to] = level[v] + 1;
                q.push(e.to);
            }
        }

        return level[sink] != -1;
    }

    // Sends flow through the level graph
    ll dfs(int v, ll pushed) {
        if (pushed == 0) {
            return 0;
        }

        if (v == sink) {
            return pushed;
        }

        for (int& cid = ptr[v];
             cid < (int)adj[v].size();
             ++cid) {

            int id = adj[v][cid];
            FlowEdge& e = edges[id];

            if (level[e.to] != level[v] + 1) {
                continue;
            }

            if (e.cap - e.flow <= 0) {
                continue;
            }

            ll tr = dfs(
                e.to,
                min(pushed, e.cap - e.flow)
            );

            if (tr == 0) {
                continue;
            }

            e.flow += tr;

            // Reverse edge
            edges[id ^ 1].flow -= tr;

            return tr;
        }

        return 0;
    }

    // Returns maximum flow
    ll max_flow() {
        ll totalFlow = 0;

        while (bfs()) {
            fill(ptr.begin(), ptr.end(), 0);

            while (true) {
```

CÓDIGO · dinic

```
                ll pushed = dfs(source, INF);

                if (pushed == 0) {
                    break;
                }

                totalFlow += pushed;
            }
        }

        return totalFlow;
    }
};

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    /*
        BASIC USAGE
        -----------

        Suppose we have nodes:

            0, 1, 2, 3

        plus:

            source = 4
            sink   = 5

        Therefore:

            totalNodes = 6
    */

    int totalNodes = 6;
    int source = 4;
    int sink = 5;

    Dinic dinic(totalNodes, source, sink);

    /*
        Add directed edges:

        add_edge(from, to, capacity)
    */

    dinic.add_edge(source, 0, 3);
    dinic.add_edge(source, 1, 2);

    dinic.add_edge(0, 2, 2);
    dinic.add_edge(0, 3, 1);
    dinic.add_edge(1, 2, 1);
    dinic.add_edge(1, 3, 2);

    dinic.add_edge(2, sink, 2);
    dinic.add_edge(3, sink, 3);

    /*
        Run Dinic.
    */

    ll flow = dinic.max_flow();

    cout << "Maximum flow: " << flow << '\n';


    /*
        ---------------------------------------
        HOW TO INSPECT THE FLOW OF EVERY EDGE
        ---------------------------------------
```

CÓDIGO · dinic

```
211
212         Every add_edge() adds TWO edges:
213
214             edges[id]     = original
215             edges[id ^ 1] = residual
216
217         Therefore, original edges are:
218
219             0, 2, 4, 6, ...
220
221         You can recover which edges were used by
222         checking:
223
224             e.flow > 0
225     */
226
227     for (int id = 0; id < (int)dinic.edges.size(); id += 2) {
228         FlowEdge& e = dinic.edges[id];
229
230         cout
231             << e.from
232             << " -> "
233             << e.to
234             << "   flow = "
235             << e.flow
236             << "/"
237             << e.cap
238             << '\n';
239     }
240
241
242     /*
243         ---------------------------------------
244         BIPARTITE MATCHING
245         ---------------------------------------
246
247         If you have:
248
249             Left side:
250                 0 ... L - 1
251
252             Right side:
253                 L ... L + R - 1
254
255             source = L + R
256             sink   = L + R + 1
257
258         then:
259
260             Dinic dinic(L + R + 2, source, sink);
261
262         source -> LEFT:
263             capacity 1
264
265         LEFT -> RIGHT:
266             capacity 1
267
268         RIGHT -> sink:
269             capacity 1
270
271
272         Example:
273
274             for (int i = 0; i < L; ++i) {
275                 dinic.add_edge(source, i, 1);
276             }
277
278             for (int j = 0; j < R; ++j) {
279                 dinic.add_edge(L + j, sink, 1);
280             }
281
282             if (left_i can match right_j) {
```

 # CÓDIGO · dinic

```
                dinic.add_edge(i, L + j, 1);
            }

            ll matching = dinic.max_flow();


        ---------------------------------------
        RECOVER MATCHES
        ---------------------------------------

        for (int id = 0;
             id < (int)dinic.edges.size();
             id += 2) {

            FlowEdge& e = dinic.edges[id];

            bool fromLeft =
                e.from >= 0 &&
                e.from < L;

            bool toRight =
                e.to >= L &&
                e.to < L + R;

            if (fromLeft && toRight && e.flow == 1) {

                int left = e.from;
                int right = e.to - L;

                cout << left << " " << right << '\n';
            }
        }


        ---------------------------------------
        B-MATCHING
        ---------------------------------------

        If every node can be matched K times,
        just change the capacities:

            source -> left    capacity K
            right  -> sink    capacity K

        while:

            left -> right     capacity 1

        Example for degree <= 2:

            dinic.add_edge(source, left, 2);
            dinic.add_edge(right, sink, 2);

        The reconstruction is exactly the same:
        left -> right edges with flow == 1.
    */

    return 0;
}
```

floydWarshall

GRAPH · C++ · 41 LÍNEAS

CÓDIGO · floydWarshall

```cpp
#include <bits/stdc++.h>

const int nax {500};
std::array< std::array<int, nax>, nax> dis{}, p{};

const int INF {1'000'000'000};
for (auto& row : dis) row.fill(INF);
for (auto& row : p) row.fill(-1);

for (int b{}; b < n; ++b) {
    for (int i{}; i < n; ++i) {
        for (int j{}; j < n; ++j) {
            if (dis[i][b] == INF || dis[b][j] == INF) continue;
            if (dis[i][j] > dis[i][b] + dis[b][j]) {
                dis[i][j] = dis[i][b] + dis[b][j];
                p[i][j] = b;
            }
        }
    }
}

int cycleSize{INF};
for (int i{}; i < n; ++i) {
    cycleSize = std::min(cycleSize, dis[i][i]);
}

std::vector<int> ans{};
for (int I{}; I < n; ++I) {
    if (dis[I][I] == cycleSize){
        auto reconstruct = [&](auto& self, int i, int j) -> void {
            if (p[i][j] == -1) {
                ans.push_back(i);
                return;
            }
            auto k {p[i][j]};
            self(self, i, k);
            self(self, k, j);
        };
        reconstruct(reconstruct, I, I);
    }
}
```

lca

GRAPH · C++ · 44 LÍNEAS

 # CÓDIGO · lca

```cpp
#include <bits/stdc++.h>

const int LOG {20};
const int INF {1'000'000'000};

std::vector<int> depth(n, INF);
depth[0] = 0;

std::vector<std::vector<int>> up(n, std::vector<int> (LOG, -1));
auto dfs = [&](auto& self, int node) -> void {
    for (auto u : adj[node]) {
        if (depth[u] != INF) continue;
        depth[u] = depth[node] + 1;
        up[u][0] = node;
        self(self, u);
    }
};
dfs(dfs, 0);
for (int k{1}; k < LOG; ++k) {
    for (int i{}; i < n; ++i) {
        int m {up[i][k - 1]};
        if (m != -1) {
            up[i][k] = up[m][k - 1];
        }
    }
}
auto lca = [&](int a, int b) {
    if (depth[a] < depth[b]) std::swap(a, b);
    int k{depth[a] - depth[b]};

    for (int i{}; i < LOG; ++i) {
        if (k & (1 << i)) {
            a = up[a][i];
        }
    }
    if (a == b) return a;
    for (int j{LOG - 1}; j >= 0; --j) {
        if (up[a][j] != up[b][j]) {
            a = up[a][j];
            b = up[b][j];
        }
    }
    return up[a][0];
}
```

mst

GRAPH · C++ · 33 LÍNEAS

CÓDIGO · mst

```cpp
// Kruskal
std::vector<std::tuple<int,int,int>> edges{};
std::ranges::sort(edges);

UF dsu{n};
std::vector<std::vector<int>> adj(n);
for (auto [weight, u, v] : edges) {
    if (dsu.unite(u, v)) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    if (dsu.components() == 1) break;
}

// PRIM
using T = std::pair<int, int>;
std::priority_queue<T, std::vector<T>, std::greater<T>> pq{};
std::vector<int> vis(n);
auto process = [&](int node) {
    vis[node] = true;
    for (auto [u, w] : adj[node]) {
        if (vis[u]) continue;
        pq.emplace(w, u);
    }
};
process(0);
long long cost{};
while (!pq.empty()) {
    auto [w, node] {pq.top()}; pq.pop();
    if (vis[node]) continue;
    cost += w;
    process(node);
}
```

stronglyConnectedComponents

GRAPH · C++ · 31 LÍNEAS

 # CÓDIGO · stronglyConnectedComponents

```cpp
const int INF {1'000'000'000};
std::vector<int> low(n), pre(n), id(n, -1);
std::stack<int> stk{};
int cont{}, numSCC{};

auto dfs = [&](auto& self, int node) -> void {
    pre[node] = low[node] = ++cont;
    stk.push(node);

    for (auto u : adj[node]) {
        if (pre[u] == 0) {
            self(self, u);
        }
        low[node] = std::min(low[node], low[u]);
    }
    if (low[node] == pre[node]) {
        while (true) {
            auto x{stk.top()}; stk.pop();
            id[x] = numSCC;
            low[x] = INF;
            if (x == node) break;
        }
        ++numSCC;
    }
};
for (int i{}; i < n; ++i) {
    if (pre[i] == 0) {
        dfs(dfs, i);
    }
}

```

SECCIÓN 03

Math

5 algoritmos

binPow

MATH · C++ · 14 LÍNEAS

CÓDIGO · binPow

```cpp
const int mod = 1e9 + 7;

auto power (long long a, long long exp) {
    long long r{1};
    while (exp) {
        if (exp % 2) {
            r = (a * r) % mod;
        }
        a = (a * a) % mod;
        exp /= 2;
    }
    return r;
}

```

combinatorics

MATH · C++ · 41 LÍNEAS

 # CÓDIGO · combinatorics

```cpp
#include <bits/stdc++.h>

long long power(long long a, long long b);
long long multi(long long a, long long b);
long long divide(long long a, long long b);


const int nax {67};
auto pascal = []() {
    std::array< std::array<long long, nax>, nax> p{};
    p[0][0] = 1;
    for (int i{1}; i < nax; ++i) {
        p[i][0] = 1;
        for (int j{1}; j < nax; ++j) {
            p[i][j] += p[i - 1][j] + p[i - 1][j - 1];
        }
    }
    return p;
}();

const int nax {100'000};

std::vector<long long> invFacto(nax, 1);
const auto facto = []() {
    std::array<long long, nax> f;
    f[0] = 1;
    for (int i{1}; i < nax; ++i) {
        f[i] = multi(f[i - 1], i);
    }
    invFacto[nax - 1] = divide(1, facto[nax - 1]);
    for (int i{nax - 1}; i > 0; --i) {
        invFacto[i - 1] = multi(invFacto[i], i);
    }
    return f;
}();

long long nCk(int n, int k) {
    if (k < 0 || n < k) return 0;
    return pascal[n][k];
    return multi(facto[n], multi(invFacto[k], invFacto[n - k]));
}
```

modArithmetic

MATH · C++ · 20 LÍNEAS

CÓDIGO · modArithmetic

```cpp
const int mod = 1e9 + 7;
long long power(long long a, long long exp);

auto toMod(long long x) {
    return ((x % mod) + mod) % mod;
}
auto sum(long long a, long long b) {
    auto ans {a + b};
    return (ans >= mod) ? ans - mod : ans;
}
auto sub(long long a, long long b) {
    auto ans {a - b};
    return (ans < 0) ? ans + mod : ans;
}
auto multi(long long a, long long b) {
    return (a * b) % mod;
}
auto divide(long long a, long long b) {
    return multi(a, power(b, mod - 2));
}
```

precision

MATH · C++ · 19 LÍNEAS

CÓDIGO · precision

```cpp
#include <bits/stdc++.h>

constexpr double eps {1e-9};

bool equal(double a, double b) {
    if (std::abs(a - b) <= eps) return true;
    return std::abs(a - b) <= std::max(std::abs(a), std::abs(b)) * eps;
}
bool less(double a, double b) {
    return a < (b - eps);
}
bool lessEqual(double a, double b) {
    return a < (b + eps);
}
long long divCeil(long long x, long long y) {
    bool neg {(x < 0) != (y < 0)};
    if (neg) return x / y;
    return (x / y) + (x % y != 0);
}
```

primes

MATH · C++ · 71 LÍNEAS

CÓDIGO · primes

```cpp
#include <bits/stdc++.h>

std::vector<long long> primes{};
const int nax {100'000};

const auto gc = []() {
    std::bitset<nax + 1> c{};
    c[0] = c[1] = true;
    primes.reserve((nax / std::log(nax)) * 1.2);

    for (int m{4}; m <= nax; m += 2) c[m] = true;
    primes.push_back(2);

    for (long long p{3}; p <= nax; p += 2) {
        if (c[p]) continue;
        primes.push_back(p);
        for (long long m{p * p}; m <= nax; m += p + p) {
            c[m] = true;
        }
    }
    return c;
}();

bool isPrime(long long m) {
    if (m <= nax) return !gc[m];
    for (auto p : primes) {
        if (p * p > m) break;
        if (m % p == 0) return false;
    }
    return true;
}

auto primeFactors(long long m) {
    std::vector<std::pair<long long,int>> f{};
    for (auto p : primes) {
        if (p * p > m) break;
        int cont{};
        while (m % p == 0) {
            ++cont;
            m /= p;
        }
        if (cont) f.emplace_back(p, cont);
    }
    if (m != 1) f.emplace_back(m, 1);
    return f;
}

auto eulerPhiSieve = []() {
    std::array <long long, nax + 1> a{};
    std::iota(a.begin(), a.end(), 0);
    for (int i{2}; i <= nax; ++i) {
        if (a[i] == i) {
            for (int j{i}; j <= nax; j += i) {
                a[j] = (a[j] / i) * (i - 1);
            }
        }
    }
    return a;
}();

auto eulerPhi(long long n) {
    if (n <= nax) return eulerPhiSieve[n];
    long long ans{n};
    for (auto p : primes) {
        if (p * p > n) break;
        if (n % p == 0) ans -= ans / p;
```

CÓDIGO · primes

```
            while (n % p == 0) n /= p;
        }
        if (n != 1) ans -= ans / n;
        return ans;
    }
```

SECCIÓN 04

Misc

2 algoritmos

jose

MISC · C++ · 22 LÍNEAS

CÓDIGO · jose

```cpp
int J(int n, int k) {
    if (n == 1) return 0; // 1 (1-index)
    return (J(n - 1, k) + k) % n;
}

int josephus(int n, int k) {
    int res = 0;
    for (int i = 1; i <= n; ++i){
        res = (res + k) % i;
    }
    return res; // + 1 (1-index)
}

std::vector<int> ans{};
int pos{1};
// 1-index
for (int i{}; i < n; ++i) {
    int r {n - i};
    pos = ((pos + k - 1) % r) + 1;
    ans.push_back(jose.find(pos));
    jose.kill(ans.back());
}
```

random

MISC · C/C++ Header · 20 LÍNEAS

CÓDIGO · random

```cpp
#ifndef RANDOM_H
#define RANDOM_H

#include <chrono>
#include <random>

namespace Random {
    inline std::mt19937_64 mt {
        static_cast<std::seed_seq::result_type> (
            std::chrono::steady_clock::now().time_since_epoch().count()
        )
    };

    template<typename T>
    T get(T min, T max) {
        return std::uniform_int_distribution<T>{min, max}(mt);
    }
}

#endif
```

SECCIÓN 05

String

4 algoritmos

hashing

STRING · C++ · 33 LÍNEAS

CÓDIGO · hashing

```cpp
// To hash a multiset: Sum of elements of: (B^num) * freq[num]

const ll maxn = 1e6 + 5;
const ll M = 1e9+9; // prime modulo (alternative: 10000000000000061 or 10000000000061)
const ll B = 131; // any number > sz(alphabet) and coprime to M.
string s,t;
ll pws[maxn],h[maxn],tams,tamt,hasht=0,ans=0;

ll conv(char c){ return (c-'a'+1); }
bool sameHash(int l1, int len1, int l2, int len2){
    int r1 = l1 + len1;
    int r2 = l2 + len2;
    ll h1 = (h[r1]-h[l1]*pws[len1]%M + M)%M;
    ll h2 = (h[r2]-h[l2]*pws[len2]%M + M)%M;
    return h1 == h2;
}
void precalc(){
    tams = sz(s), tamt = sz(t);
    pws[0] = 1;
    fore(i,1,maxn) pws[i] = (pws[i-1]*B)%M;
    h[0] = conv(s[0]);
    fore(i,0,tams) h[i+1] = ((h[i]*B)+conv(s[i]))%M;
    fore(i,0,tamt) hasht = ((hasht*B)+conv(t[i]))%M;
}
void doit(){
    cin>>s; //main text.
    cin>>t; //pattern.
    precalc();
    fore(i,tamt,tams+1){ // Check occurrences of t in s
        ll cur_hash = (h[i]-h[i-tamt]*pws[tamt]%M + M)%M;
        if (cur_hash == hasht) ans++;
    }
}
```

kmp

STRING · C++ · 40 LÍNEAS

CÓDIGO · kmp

```cpp
vector<long long> prefix_function(string s){
    long long n= (int)s.size();
    vector<long long> pi(n);

    for(int i = 1; i<n; i++){
        long long j=pi[i-1];

        while(j>0 && s[i]!=s[j])
            j=pi[j-1];

        if(s[i]==s[j])
            j++;

        pi[i]=j;
    }

    return pi;
}

vector<long long> kmp(string s,string pat){
    vector<long long> pi=prefix_function(pat);
    vector<long long> ans;

    long long j=0;

    for(int i = 0; i< (int)s.size(); i++){
        while(j>0 && s[i]!=pat[j])
            j=pi[j-1];

        if(s[i]==pat[j])
            j++;

        if(j==sz(pat)){
            ans.pb(i-sz(pat)+1);
            j=pi[j-1];
        }
    }

    return ans;
}
```

Trie

STRING · C++ · 64 LÍNEAS

CÓDIGO · Trie

```cpp
struct Trie {
    struct Node {
        int nxt[26];
        bool end;

        Node() {
            fill(nxt, nxt + 26, -1);
            end = false;
        }
    };

    vector<Node> tr;

    Trie() {
        tr.emplace_back();
    }

    void insert(const string& s) {
        int u = 0;

        for (char c : s) {
            int x = c - 'a';

            if (tr[u].nxt[x] == -1) {
                tr[u].nxt[x] = (int)tr.size();
                tr.emplace_back();
            }

            u = tr[u].nxt[x];
        }

        tr[u].end = true;
    }

    bool find(const string& s) {
        int u = 0;

        for (char c : s) {
            int x = c - 'a';

            if (tr[u].nxt[x] == -1)
                return false;

            u = tr[u].nxt[x];
        }

        return tr[u].end;
    }

    bool prefix(const string& s) {
        int u = 0;

        for (char c : s) {
            int x = c - 'a';

            if (tr[u].nxt[x] == -1)
                return false;

            u = tr[u].nxt[x];
        }

        return true;
    }
};
```

z_funtion

STRING · C++ · 21 LÍNEAS

CÓDIGO · z_funtion

```cpp
vector<long long> z_function(string s){
    ll n=(int)s.size();
    vector<long long> z(n);
    long long l=0,r=0;

    for(int i = 1; i<n ; i++){
        if(i<=r)
            z[i]=min(r-i+1,z[i-l]);

        while(i+z[i]<n && s[z[i]]==s[i+z[i]])
            z[i]++;

        if(i+z[i]-1>r){
            l=i;
            r=i+z[i]-1;
        }
    }

    return z;
}

```